

PROJECT

Arduino Mega Password-Protected Lightning Stairs System

Project Overview

This project uses an Arduino Mega powered through a laptop's USB port to manage a passcode-protected staircase lighting system and a servo-controlled gate. Five IR sensors detect a person's position on the stairs, four LED strip sections (or bulbs) light up sequentially, and a 16×2 I²C LCD shows status messages. A 4×4 keypad collects the password, and a hobby servo opens/closes the gate.

Required Components

- • Arduino Mega 2560 (powered via USB from laptop)
- • 16×2 I²C LCD (PCF8574, address 0x27)
- • 4×4 Matrix Keypad
- • MG996R or SG90 Servo Motor (5 V)
- • 5 IR Obstacle Sensors (digital output LOW = beam broken)
- • 4 High-power LEDs / LED strip channels (on D10 – D13) with appropriate current-limiting resistors
- • Jumper wires, breadboard or screw terminals

Pin Connections

Module / Signal	Arduino Mega Pin	Notes	Power
Keypad ROW0	D2	Top row	--
Keypad ROW1	D3		--
Keypad ROW2	D4		--
Keypad ROW3	D5		--
Keypad COL0	D6	Left column	--
Keypad COL1	D7		--
Keypad COL2	D8		--
Keypad COL3	D9		--
LED 1	D10	Lower stair light	Common GND, +5 V via resistor
LED 2	D11	2nd-step light	"
LED 3	D12	3rd-step light	"
LED 4	D13	Top step light	"
IR Sensor 1 (LOW when blocked)	A0	Bottom step	+5 V, GND
IR Sensor 2	A1		+5 V, GND
IR Sensor 3	A2		+5 V, GND
IR Sensor 4	A3		+5 V, GND
IR Sensor 5	A4	Top step	+5 V, GND
Servo PWM	D14 (TX3)	Gate arm	+5 V, GND (≈ 500 mA stall)
LCD SDA	D20 (SDA)	I ² C data	+5 V, GND
LCD SCL	D21 (SCL)	I ² C clock	+5 V, GND

⚡ ****Power note:**** The entire circuit is powered from the laptop's USB 5 V rail (≈ 500 mA available). Ensure your servo's stall current is below this limit or run it briefly to avoid overcurrent. LEDs should be low-current or driven via a transistor if needed.

Arduino Sketch

The full, formatted source code is included below. Upload it to the Arduino Mega via the Arduino IDE.

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
```

```

#include <Keypad.h>
#include <Servo.h>

// LCD
LiquidCrystal_I2C lcd(0x27, 16, 2);

// Servo
Servo gateServo;
int servoPin = 14;
unsigned long gateOpenTime = 0;
bool gateOpen = false;
bool gateByStarKey = false;

// Keypad
const byte ROWS = 4;
const byte COLS = 4;
char keys[ROWS][COLS] = {
  {'1','2','3','A'},
  {'4','5','6','B'},
  {'7','8','9','C'},
  {'*','0','#','D'}
};
byte rowPins[ROWS] = {2, 3, 4, 5};
byte colPins[COLS] = {6, 7, 8, 9};
Keypad keypad = Keypad(makeKeymap(keys), rowPins, colPins, ROWS, COLS);

// IR Sensors and LEDs
int irPins[5] = {A0, A1, A2, A3, A4};
int bulbPins[4] = {10, 11, 12, 13};
unsigned long ledTimers[4] = {0, 0, 0, 0};
bool ledStates[4] = {false, false, false, false};
const unsigned long ledDelay = 2000; // 2 seconds

// Password
String password = "1234";
String input = "";
bool accessGranted = false;
bool stairsActive = false;
bool systemResetRequested = false;

void setup() {
  Serial.begin(9600);
  lcd.init();
  lcd.backlight();
  lcd.setCursor(0, 0);
  lcd.print("Enter Password:");

  gateServo.attach(servoPin);
  gateServo.write(120); // Closed

  for (int i = 0; i < 5; i++) pinMode(irPins[i], INPUT);
  for (int i = 0; i < 4; i++) {
    pinMode(bulbPins[i], OUTPUT);
    digitalWrite(bulbPins[i], LOW);
  }
}

void loop() {
  char key = keypad.getKey();
  if (key) {
    if (!accessGranted) {
      handlePassword(key);
    }
  }
}

```

```

    } else {
        if (key == '*') {
            stairsActive = false;
            lcd.clear();
            lcd.print("Stairs Deactivated");
            turnOffAllLEDs();
            gateServo.write(20); // Open gate
            gateOpenTime = millis();
            gateByStarKey = true;
            gateOpen = true;
            systemResetRequested = true;
        }
    }
}

// Handle auto-close gate after 10s (normal) or 5s (* key)
if (gateOpen) {
    if ((gateByStarKey && millis() - gateOpenTime >= 5000) || (!gateByStarKey && millis() - gateOpenTime >= 10000)) {
        gateServo.write(120); // Close gate
        gateOpen = false;
        gateByStarKey = false;

        if (systemResetRequested) {
            lcd.clear();
            lcd.print("Enter Password:");
            accessGranted = false;
            input = "";
            stairsActive = false;
            systemResetRequested = false;
        }
    }
}

if (stairsActive) {
    handleIRSensors();
    updateLEDStates();
}
}

void handlePassword(char key) {
    if (key == '#') {
        if (input == password) {
            lcd.clear();
            lcd.print("Access Granted");
            gateServo.write(20);
            accessGranted = true;
            stairsActive = true;
            gateOpen = true;
            gateOpenTime = millis();
            delay(1000);
            lcd.clear();
            lcd.print("Stairs Activated");
        } else {
            lcd.clear();
            lcd.print("Wrong Password");
            delay(1500);
            lcd.clear();
            lcd.print("Enter Password:");
        }
        input = "";
    } else if (key == '*') {

```

```

        input = "";
        lcd.setCursor(0, 1);
        lcd.print("          ");
    } else {
        if (input.length() < 16) {
            input += key;
            lcd.setCursor(0, 1);
            lcd.print(input);
        }
    }
}

void handleIRSensors() {
    if (digitalRead(irPins[0]) == LOW) {
        lightLED(0);
    } else if (digitalRead(irPins[1]) == LOW) {
        lightLED(0);
        lightLED(1);
    } else if (digitalRead(irPins[2]) == LOW) {
        lightLED(0);
        lightLED(1);
        lightLED(2);
    } else if (digitalRead(irPins[3]) == LOW) {
        lightLED(2);
        lightLED(3);
        lightLED(1);
    } else if (digitalRead(irPins[4]) == LOW) {
        lightLED(3);
        lightLED(2);
    }
}

void lightLED(int index) {
    digitalWrite(bulbPins[index], HIGH);
    ledTimers[index] = millis();
    ledStates[index] = true;
}

void updateLEDStates() {
    for (int i = 0; i < 4; i++) {
        if (ledStates[i] && (millis() - ledTimers[i] > ledDelay)) {
            digitalWrite(bulbPins[i], LOW);
            ledStates[i] = false;
        }
    }
}

void turnOffAllLEDs() {
    for (int i = 0; i < 4; i++) {
        digitalWrite(bulbPins[i], LOW);
        ledStates[i] = false;
    }
}

```